

SQL Interview Prep

Junior .NET Developer Edition

50+ Questions & Answers with Code Examples

This guide covers the most common SQL interview questions for Junior .NET Developer roles. Topics include basic queries, JOINS, aggregations, indexes, stored procedures, transactions, and performance tips. Study each section and practice writing the queries by hand.

■ Section 1 – SQL Basics

Q1. What is SQL and what is it used for?

SQL (Structured Query Language) is the standard language for managing and manipulating relational databases. It is used to query data (SELECT), insert rows (INSERT), update records (UPDATE), delete data (DELETE), and define database structure (CREATE, ALTER, DROP).

■ *Tip: In .NET projects you will usually use SQL with ADO.NET, Dapper, or Entity Framework.*

Q2. What are the main SQL command categories?

SQL commands are grouped into five categories: • DDL (Data Definition Language): CREATE, ALTER, DROP, TRUNCATE • DML (Data Manipulation Language): SELECT, INSERT, UPDATE, DELETE • DCL (Data Control Language): GRANT, REVOKE • TCL (Transaction Control Language): COMMIT, ROLLBACK, SAVEPOINT • DQL (Data Query Language): SELECT

Q3. What is the difference between WHERE and HAVING?

WHERE filters rows BEFORE grouping (works on individual rows). HAVING filters groups AFTER a GROUP BY clause (works on aggregated results).

```
SELECT DepartmentID, COUNT(*) AS EmpCount
FROM Employees
WHERE Salary > 3000
GROUP BY DepartmentID
HAVING COUNT(*) > 5;
```

■ *Tip: WHERE cannot use aggregate functions; HAVING can.*

Q4. What is the difference between DELETE, TRUNCATE, and DROP?

DELETE removes specific rows and can be rolled back (DML). TRUNCATE removes all rows quickly, resets identity, and cannot be rolled back easily (DDL). DROP removes the entire table including its

structure (DDL).

■ *Tip: Use **TRUNCATE** when you need to clear a table fast in dev/test environments.*

Q5. What is a PRIMARY KEY?

A PRIMARY KEY uniquely identifies each row in a table. It cannot be NULL and must be unique. A table can have only one primary key, but it can be composite (multiple columns).

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT NOT NULL  
);
```

Q6. What is a FOREIGN KEY?

A FOREIGN KEY is a column (or set of columns) that creates a link between two tables. It references the PRIMARY KEY of another table and enforces referential integrity.

```
ALTER TABLE Orders  
ADD CONSTRAINT FK_Customer  
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID);
```

Q7. What is the difference between CHAR and VARCHAR?

CHAR(n) stores fixed-length strings — always uses n bytes regardless of actual content. VARCHAR(n) stores variable-length strings — uses only the bytes needed (plus 2 bytes overhead). Use CHAR for fixed values like country codes; VARCHAR for names, emails, etc.

Q8. What is NULL in SQL and how do you check for it?

NULL represents a missing or unknown value. You cannot use = NULL; instead use IS NULL or IS NOT NULL.

```
SELECT * FROM Employees WHERE ManagerID IS NULL;
```

■ *Tip: **NULL != NULL** in SQL — two NULLs are never equal.*

■ Section 2 – JOINS

Q9. What are the types of JOINS in SQL?

INNER JOIN – returns rows that have matching values in both tables. LEFT JOIN – returns all rows from the left table, and matching rows from the right (NULL if no match). RIGHT JOIN – opposite of LEFT JOIN. FULL OUTER JOIN – returns all rows from both tables (NULL where no match). CROSS JOIN – returns the Cartesian product of both tables.

Q10. Write a query using INNER JOIN.

Get all orders with the customer name:

```
SELECT o.OrderID, c.CustomerName, o.OrderDate
FROM Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID;
```

Q11. What is the difference between INNER JOIN and LEFT JOIN?

INNER JOIN returns only matching rows. LEFT JOIN returns all rows from the left table even if there is no match in the right table — unmatched right-side columns will be NULL.

```
-- Employees without a department:
SELECT e.Name, d.DeptName
FROM Employees e
LEFT JOIN Departments d ON e.DeptID = d.DeptID
WHERE d.DeptID IS NULL;
```

Q12. What is a SELF JOIN?

A SELF JOIN joins a table to itself. Useful for hierarchical data like employee-manager relationships.

```
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

Q13. What is the difference between UNION and UNION ALL?

UNION combines result sets and removes duplicate rows. UNION ALL keeps all rows including duplicates. UNION ALL is faster because it skips the deduplication step.

```
SELECT Name FROM Customers
UNION ALL
SELECT Name FROM Suppliers;
```

■ *Tip: Both queries in a UNION must have the same number of columns and compatible data types.*

■ Section 3 – Aggregation & Grouping

Q14. What aggregate functions does SQL provide?

COUNT() – number of rows. SUM() – total of a numeric column. AVG() – average value. MIN() / MAX() – smallest or largest value. These are used with GROUP BY.

```
SELECT DepartmentID,  
       COUNT(*) AS Headcount,  
       AVG(Salary) AS AvgSalary,  
       MAX(Salary) AS TopSalary  
FROM Employees  
GROUP BY DepartmentID;
```

Q15. How do you find duplicate records in a table?

Group by the column(s) you suspect have duplicates and use HAVING COUNT(*) > 1.

```
SELECT Email, COUNT(*) AS Cnt  
FROM Customers  
GROUP BY Email  
HAVING COUNT(*) > 1;
```

Q16. How do you get the second highest salary?

Use a subquery with MAX, or use the OFFSET-FETCH / ROW_NUMBER approach.

```
-- Subquery approach:  
SELECT MAX(Salary) FROM Employees  
WHERE Salary < (SELECT MAX(Salary) FROM Employees);  
  
-- ROW_NUMBER approach (SQL Server):  
SELECT Salary FROM (  
    SELECT Salary, ROW_NUMBER() OVER (ORDER BY Salary DESC) AS rn  
    FROM Employees) t  
WHERE rn = 2;
```

■ *Tip: Interviewers love this question — know both approaches!*

Q17. What is the difference between COUNT(*) and COUNT(column)?

COUNT(*) counts all rows including NULLs. COUNT(column) counts only non-NULL values in that column.

■ Section 4 – Subqueries & CTEs

Q18. What is a subquery?

A subquery is a SELECT statement nested inside another SQL statement. It can appear in SELECT, FROM, or WHERE clauses.

```
SELECT Name FROM Employees  
WHERE DeptID = (  
    SELECT DeptID FROM Departments WHERE DeptName = 'IT'  
);
```

Q19. What is a CTE (Common Table Expression)?

A CTE is a named temporary result set defined with the WITH keyword. It improves readability and can be referenced multiple times in the same query.

```
WITH HighEarners AS (  
    SELECT * FROM Employees WHERE Salary > 8000  
)  
SELECT Name, Salary FROM HighEarners  
ORDER BY Salary DESC;
```

■ *Tip: Prefer CTEs over subqueries for readability in complex queries.*

Q20. What is the difference between a correlated and non-correlated subquery?

A non-correlated subquery runs independently and its result is used by the outer query. A correlated subquery references columns from the outer query and is re-executed for each outer row (slower but sometimes necessary).

```
-- Correlated: employees earning above their dept average  
SELECT Name, Salary, DeptID  
FROM Employees e  
WHERE Salary > (  
    SELECT AVG(Salary) FROM Employees  
    WHERE DeptID = e.DeptID  
);
```

Q21. What is EXISTS vs IN?

IN checks if a value is in a list or subquery result. EXISTS checks whether a subquery returns any rows. EXISTS is generally faster when the subquery returns many rows because it stops at the first match.

```
-- EXISTS example:  
SELECT Name FROM Customers c  
WHERE EXISTS (  
    SELECT 1 FROM Orders o WHERE o.CustomerID = c.CustomerID  
);
```

■ Section 5 – Indexes & Performance

Q22. What is an index and why use it?

An index is a database object that speeds up data retrieval on a table column. Without an index, SQL performs a full table scan. With an index, it uses a B-tree structure to find rows quickly — similar to a book's index.

```
CREATE INDEX IX_Employees_Email ON Employees(Email);
```

■ *Tip: Indexes speed up reads but slow down INSERT/UPDATE/DELETE because the index must be maintained.*

Q23. What is the difference between clustered and non-clustered indexes?

A clustered index sorts and stores the actual data rows in the table in index order. A table can have only ONE clustered index (usually the primary key). A non-clustered index is a separate structure that stores pointers to the data rows. A table can have many non-clustered indexes.

■ *Tip: Primary keys create clustered indexes by default in SQL Server.*

Q24. What is a covering index?

A covering index includes all the columns needed by a query, so SQL Server can satisfy the query from the index alone without accessing the base table (no key lookup).

```
CREATE INDEX IX_Cover ON Orders(CustomerID)
INCLUDE (OrderDate, TotalAmount);
```

Q25. When should you NOT add an index?

Avoid indexes on: small tables (full scan is faster), columns rarely used in WHERE/JOIN/ORDER BY, columns with very low cardinality (e.g., a boolean IsActive column), and tables with very heavy write operations where index maintenance overhead hurts performance.

■■ Section 6 – Stored Procedures, Views & Functions

Q26. What is a stored procedure?

A stored procedure is a precompiled set of SQL statements stored in the database. Benefits: reusability, security (grant EXECUTE without direct table access), reduced network traffic, and better performance (cached execution plan).

```
CREATE PROCEDURE GetEmployeesByDept
    @DeptID INT
AS
BEGIN
    SELECT Name, Salary
    FROM Employees
    WHERE DeptID = @DeptID
END;
```

Q27. What is the difference between a stored procedure and a function?

A function MUST return a value and can be used inside a SELECT statement. A stored procedure may or may not return values and is called with EXEC. Functions cannot modify database state (no INSERT/UPDATE/DELETE); stored procedures can.

Q28. What is a VIEW?

A VIEW is a virtual table based on a SELECT query. It does not store data itself. Views simplify complex queries, provide security by exposing only certain columns, and present data in a customised format.

```
CREATE VIEW ActiveEmployees AS
SELECT EmployeeID, Name, DeptID
FROM Employees
WHERE IsActive = 1;
```

Q29. What is the difference between a VIEW and a CTE?

A VIEW is a permanent database object reusable across sessions and queries. A CTE is temporary — it exists only within a single query execution.

■ Section 7 – Transactions & ACID

Q30. What is a transaction?

A transaction is a unit of work that is performed as a whole — either all statements succeed (COMMIT) or all are undone (ROLLBACK). Transactions ensure data consistency.

```
BEGIN TRANSACTION;  
  UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 1;  
  UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 2;  
COMMIT;
```

Q31. What are the ACID properties?

Atomicity – all operations in a transaction succeed or all fail. Consistency – the database moves from one valid state to another. Isolation – concurrent transactions do not interfere with each other. Durability – committed transactions survive system failures.

■ *Tip: Interviewers often ask you to explain ACID — memorise these four!*

Q32. What is a deadlock?

A deadlock occurs when two transactions each hold a lock that the other needs, causing both to wait forever. SQL Server detects deadlocks and automatically kills one transaction (the deadlock victim).

■ *Tip: To avoid deadlocks: always access tables in the same order, keep transactions short.*

Q33. What are transaction isolation levels?

READ UNCOMMITTED – can read dirty (uncommitted) data. READ COMMITTED (default) – only reads committed data. REPEATABLE READ – rows read cannot change during the transaction. SERIALIZABLE – fully isolated; no phantom reads. SNAPSHOT – reads a version of data as it was at the start of the transaction.

■ Section 8 – Window Functions

Q34. What are window functions?

Window functions perform calculations across a set of rows related to the current row (the 'window') without collapsing rows like GROUP BY does. They use the OVER() clause.

```
SELECT Name, Salary,  
       AVG(Salary) OVER (PARTITION BY DeptID) AS DeptAvg,  
       ROW_NUMBER() OVER (ORDER BY Salary DESC) AS Rank  
FROM Employees;
```

Q35. What is the difference between ROW_NUMBER, RANK, and DENSE_RANK?

ROW_NUMBER() – assigns a unique sequential number with no gaps or ties. RANK() – assigns the same rank to ties, then skips numbers (1,1,3). DENSE_RANK() – assigns the same rank to ties but never skips numbers (1,1,2).

```
SELECT Name, Salary,  
       ROW_NUMBER() OVER (ORDER BY Salary DESC) AS RowNum,  
       RANK() OVER (ORDER BY Salary DESC) AS Rnk,  
       DENSE_RANK() OVER (ORDER BY Salary DESC) AS DenseRnk  
FROM Employees;
```

Q36. What is LAG and LEAD?

LAG accesses data from a previous row in the result set. LEAD accesses data from a following row. Both are useful for calculating differences between consecutive rows.

```
SELECT OrderDate, TotalAmount,  
       LAG(TotalAmount) OVER (ORDER BY OrderDate) AS PrevAmount  
FROM Orders;
```

■ Section 9 – SQL in .NET Context

Q37. What is the difference between ADO.NET, Dapper, and Entity Framework?

ADO.NET – low-level; you write raw SQL and manage connections manually. Maximum control, most verbose. Dapper – lightweight micro-ORM; you write SQL but it maps results to C# objects automatically. Entity Framework – full ORM; generates SQL from LINQ queries; less SQL writing but more abstraction.

■ *Tip: For interviews, know when each is appropriate. Dapper is great for performance-critical reads.*

Q38. What is a SQL injection and how do you prevent it in .NET?

SQL injection is an attack where malicious SQL is inserted into a query through user input, potentially reading or destroying data. In .NET, prevent it by always using parameterized queries or stored procedures — never concatenate user input into SQL strings.

```
// VULNERABLE:
string sql = "SELECT * FROM Users WHERE Name = '" + input + "';";

// SAFE (parameterized):
using var cmd = new SqlCommand("SELECT * FROM Users WHERE Name = @Name", conn);
cmd.Parameters.AddWithValue("@Name", input);
```

■ *Tip: SQL injection is a top interview topic for any developer working with databases.*

Q39. What is connection pooling?

Connection pooling reuses existing database connections instead of creating a new one for each request. In .NET, ADO.NET uses connection pooling automatically. It greatly improves performance under load.

Q40. What is the N+1 query problem?

The N+1 problem occurs when you execute 1 query to get a list of N items, then run N additional queries to fetch related data for each item. Solution: use a JOIN or eager loading (Include() in EF) to get all data in one query.

■ *Tip: Entity Framework's lazy loading can silently cause N+1 — always check the generated SQL.*

■ Quick Reference – Common SQL Commands

Command	Purpose	Example
SELECT	Retrieve data	SELECT * FROM Employees
INSERT	Add new row	INSERT INTO T(Col) VALUES(1)
UPDATE	Modify rows	UPDATE T SET Col=2 WHERE ID=1
DELETE	Remove rows	DELETE FROM T WHERE ID=1
TRUNCATE	Remove all rows fast	TRUNCATE TABLE Logs
INNER JOIN	Matching rows	ON a.ID = b.ID
LEFT JOIN	All left + matching right	ON a.ID = b.ID
GROUP BY	Group for aggregation	GROUP BY DeptID
HAVING	Filter groups	HAVING COUNT(*) > 5
ORDER BY	Sort results	ORDER BY Salary DESC
TOP / LIMIT	Limit rows returned	SELECT TOP 10 * FROM T
DISTINCT	Remove duplicates	SELECT DISTINCT City FROM T
CASE WHEN	Conditional logic	CASE WHEN Sal>5000 THEN 'High'
COALESCE	Return first non-NULL	COALESCE(Phone, 'N/A')

Good luck with your interview! Practice writing every query by hand — that is the best preparation.
SQL Interview Prep · Junior .NET Developer Edition